



Teoria e Aplicações de Inteligência Computacional I

Aula 6 – Introdução aos Fundamentos de Inteligência
Computacional

Algoritmos Genéticos e Computação Evolutiva

Agenda da Aula



Busca Local e Hill-Climbing

Espaço de estados · Busca sem gradiente · Problema das 9 Rainhas · Ótimos locais



Local Beam Search

k estados simultâneos · Geração de sucessores · Avaliação em pool · Conexão com AGs



Fundamentos dos Algoritmos Genéticos

Seleção natural · Analogia biológica · Cromossomo · Gene · Fitness · Fluxograma do AG



Seleção

Roleta · Torneio · Ranking · Comparação dos métodos · Maximização × Minimização



Cruzamento (Crossover)

Ponto único · Múltiplos pontos · Uniforme · BLX- α · Aritmético



Mutação

Bit-flip (binário) · Gaussiana (real) · Probabilidade P_m · Preservação da diversidade



Ciclo Geracional e Parâmetros

Nova população · Elitismo · Modelo geracional · Steady-state · N , P_o , P_m , k , σ , E



Aplicação: Sintonia de PID com AG

Planta $G(s)$ · Controlador PID · Cromossomo [K_p , K_i , K_d] · Fitness IAE · Implementação Python · Convergência

Algoritmos Genéticos

e Computação Evolutiva

*Sistemas inteligentes reais dependem de algoritmos capazes de realizar sintonia de parâmetros em superfícies de busca complexas, **frequentemente não-diferenciáveis***

O Espaço de Estados e a Busca por Ótimos

Sistemas reais de engenharia exigem a localização de parâmetros ótimos em superfícies de custo complexas.



Espaço de Estados (S)

Conjunto de todas as configurações ou vetores de parâmetros possíveis para resolver um problema físico ou de engenharia.



Função Objetivo $f: S \rightarrow \mathbb{R}$

Mapeamento quantitativo que avalia o desempenho de cada estado candidato — também chamado de paisagem de aptidão (*fitness landscape*).



Meta de Otimização

Localizar o vetor de parâmetros globalmente ótimo $\mathbf{x}^* \in S$ tal que: $\mathbf{x}^* = \operatorname{argmax} f(\mathbf{x})$ ou $\mathbf{x}^* = \operatorname{argmin} E(\mathbf{x})$, para maximização ou minimização, respectivamente.

$$\mathbf{x}^* = \operatorname{argmax}_{\mathbf{x} \in S} f(\mathbf{x}) \quad \text{ou} \quad \mathbf{x}^* = \operatorname{argmin}_{\mathbf{x} \in S} E(\mathbf{x})$$

O Gargalo das Derivadas

Métodos clássicos como a Descida do Gradiente e o Backpropagation impõem restrições severas sobre a superfície de erro — limitando sua aplicabilidade em problemas reais de engenharia.



Exigência de Diferenciabilidade

A superfície de erro deve ser contínua, diferenciável e derivável em todo o domínio — condição nem sempre satisfeitas em sistemas reais.



Sensibilidade a Ruídos

Pequenas perturbações ou ruídos de amostragem geram derivadas numéricas ruidosas, fazendo o otimizador divergir ou oscilar sem convergir.



Inviabilidade em Espaços Discretos

Completamente inaplicável em espaços de parâmetros discretos, categóricos ou governados por restrições lógicas não-lineares — onde derivadas não existem.

Esses gargalos motivam o uso de métodos livres de gradiente — como algoritmos evolutivos e meta-heurísticas — capazes de operar em superfícies arbitrárias.

Otimização sem Histórico de Trajetória

Algoritmos indicados para cenários onde apenas o estado final — a configuração ótima — é relevante. O caminho percorrido ou a árvore de decisões passadas não precisa ser armazenado em memória.



Eficiência de Memória $O(1)$

Consumo de memória RAM insignificante — $O(1)$ para estados individuais — sendo altamente escalável mesmo para espaços de busca de alta dimensão.



Problema das 9 Rainhas

Posicionar 9 rainhas em um tabuleiro 9×9 sem conflitos — apenas o estado final importa, não a ordem de posicionamento.



Aplicações em Engenharia

Alocação de cargas em sistemas de energia e sintonia de controladores PID e nebulosos — onde apenas os parâmetros finais importam.

A Abordagem Gulosa Simples

Funciona de forma funcionalmente análoga a uma **descida de gradiente discretizada** — porém sem exigir diferenciabilidade da superfície de custo.



Mecânica Gulosa

Avalia o estado corrente, gera seus vizinhos diretos e move-se estritamente para o vizinho que apresenta a melhoria mais acentuada de fitness — sem retrocesso.



Critério de Parada

Interrompe a execução imediatamente se nenhum vizinho direto superar a qualidade do estado atual — convergindo para um ótimo local ou global.



Analogia com Gradiente Discreto

Equivalente a uma descida de gradiente discretizada: em vez de calcular $\nabla f(x)$, avalia f nos vizinhos e seleciona a direção de maior melhoria.

Busca Gulosa Local

A partir do estado atual, avalia os estados vizinhos e escolhe aquele que apresenta a melhor melhoria imediata.



Regra de Decisão

Se existe um vizinho melhor → move-se para ele. Se não existe vizinho melhor → a busca é encerrada.



Comportamento Guloso

Busca continuamente melhorar a solução, mas nunca aceita uma piora temporária – sem retrocesso.



Limitações Críticas

Pode ficar preso em um **máximo local, platô ou crista**, mesmo existindo uma solução melhor em outra região do espaço de busca.



Dependência do Estado Inicial

O resultado pode variar significativamente conforme o ponto de partida – reinicializações aleatórias são uma estratégia comum.

O Problema das 9 Rainhas

Posicionar 9 rainhas em um tabuleiro 9×9 de forma que nenhuma ataque outra — sem importar a ordem em que foram colocadas.



Representação do Estado

Cada estado é uma **configuração completa** das 9 rainhas no tabuleiro (uma por coluna)

Espaço de Estados: $9^9 = 387.420.489$ configurações possíveis



Função Objetivo (h)

Número de pares de rainhas em **conflito** (ataque mútuo)

Meta: encontrar o estado com **$h = 0$** (nenhum conflito)



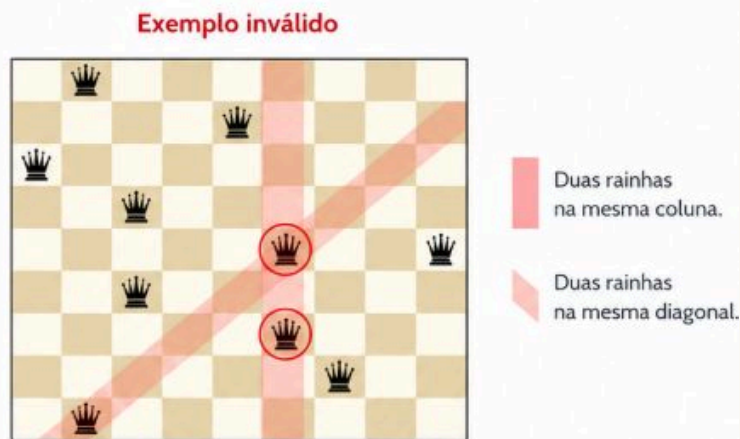
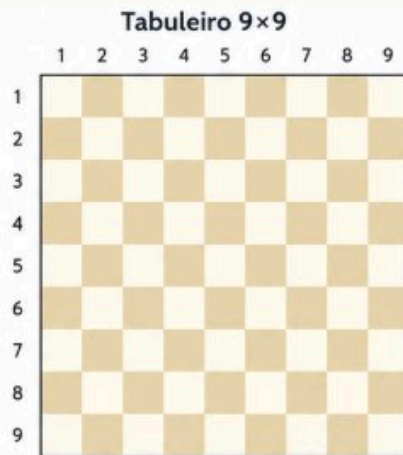
Busca Local

A partir do estado atual, move-se para o **vizinho com menor h**, sem guardar o caminho percorrido

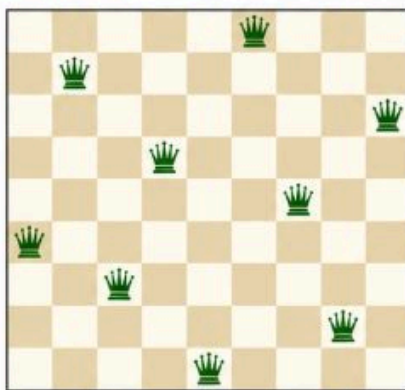
Vantagem: **memória constante** — apenas o estado atual é armazenado

Restrições do Problema das 9 Rainhas

Posicionar 9 rainhas em um tabuleiro 9×9 de forma que nenhuma rainha ataque outra — nenhuma pode estar na mesma linha, coluna ou diagonal.



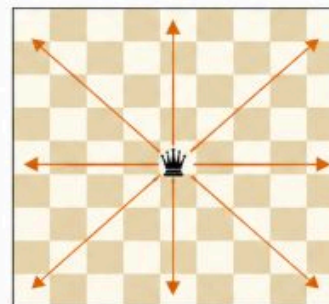
Exemplo válido (uma das 352 soluções possíveis)



Restrições do problema

- 1 rainha por linha
- 1 rainha por coluna
- 1 rainha por diagonal principal (\\)
- 1 rainha por diagonal secundária (/)

Como uma rainha se move (ataques possíveis)



- O objetivo é encontrar uma configuração como o exemplo válido, onde nenhuma rainha ameaça a outra.
- **Espaço Bruto de Configurações**
 - **Sem restrições** espaço de Estados: $9^9 = 387.420.489$ configurações possíveis

Definindo a Função de Avaliação $h(s)$

Para aplicar o **Hill-Climbing**, precisamos de uma função que quantifique a qualidade de cada estado — quanto menor $h(s)$, melhor a solução.

$h(s)$ = número de pares de rainhas que se atacam mutuamente



$h = 20$

Solução ruim — muitos conflitos entre rainhas



$h = 5$

Solução melhor — poucos conflitos restantes



$h = 1$

Solução muito boa — apenas um par em conflito



$h = 0$

Solução perfeita — nenhum par de rainhas se ataca

 O algoritmo Hill-Climbing move-se sempre para o estado vizinho com menor h , parando quando nenhum vizinho melhora o valor atual.

Estado Atual e Geração de Vizinhos

A partir do estado atual, o algoritmo gera todos os estados vizinhos movendo uma rainha para outra coluna e avalia qual deles minimiza $h(s)$.

Estado atual: $S = [2, 5, 8, 1, 4, 7, 3, 6, 9]$

Cada posição representa uma linha; o número representa a coluna da rainha naquela linha.



Operação de Vizinhança

Mover uma rainha para outra coluna dentro da mesma linha — uma alteração por vez no vetor de estado.



Exemplos de Vizinhos Gerados

$S_1 = [3, 5, 8, 1, 4, 7, 3, 6, 9]$ → rainha da linha 1 move para coluna 3

$S_2 = [2, 6, 8, 1, 4, 7, 3, 6, 9]$ → rainha da linha 2 move para coluna 6

$S_3 = [2, 5, 7, 1, 4, 7, 3, 6, 9]$ → rainha da linha 3 move para coluna 7



Avaliação dos Vizinhos

Para cada vizinho gerado, calcula-se $h(S_n)$. O algoritmo move-se para o vizinho com menor h — se for menor que o estado atual.

 Para 9 rainhas, cada estado tem $9 \times 8 = 72$ vizinhos possíveis (8 colunas alternativas por rainha $\times$ 9 rainhas).

Calculando o Custo de Cada Vizinho

O algoritmo avalia $h(s)$ para cada vizinho gerado e seleciona aquele com menor número de conflitos — desde que seja melhor que o estado atual.

Estado	Conflitos $h(s)$
Atual	8
Vizinho 1	6
Vizinho 2	10
Vizinho 3	5 ← melhor
Vizinho 4	7
Vizinho 5	9

Meu estado atual tem 8 conflitos. O melhor vizinho tem 5 conflitos (Vizinho 3). Como $5 < 8$, movo-me para S_3 .

$$S \leftarrow S_3$$

 O processo se repete: S_3 torna-se o novo estado atual e novos vizinhos são gerados e avaliados — até que nenhum vizinho melhore $h(s)$.

O Problema do Máximo Local

O Hill-Climbing pode convergir para uma solução aparentemente boa, mas incapaz de alcançar a solução ótima global — pois nunca aceita uma piora temporária.



Cenário: Ótimo Local

O algoritmo percorre: $h=8 \rightarrow h=6 \rightarrow h=4 \rightarrow h=2 \rightarrow h=1$ (estado atual).
Todos os vizinhos têm custo maior: $h(S_1)=2$, $h(S_2)=3$, $h(S_3)=2$.
Como nenhum vizinho melhora $h=1$, o algoritmo para.



A Armadilha

Pode existir uma solução perfeita $h=0$ em outra região do espaço de busca. Para alcançá-la, seria necessário aceitar uma piora temporária de $h=1$ para $h=2$ — o que o Hill-Climbing tradicional recusa.

O Hill-Climbing é míope: enxerga apenas os vizinhos imediatos e nunca aceita retroceder, mesmo que isso leve a uma solução melhor.



Soluções: Hill-Climbing com Reinício Aleatório, Simulated Annealing e Algoritmos Genéticos — que exploram o espaço de busca de forma mais ampla.

Quando a Busca Local Não é Suficiente

O Hill-Climbing é míope: considera apenas os vizinhos do estado atual e pode ficar preso em um ótimo local — sem garantia de encontrar a solução globalmente ótima.



Reinício Aleatório

Executar a busca repetidamente a partir de diferentes estados iniciais, aumentando a probabilidade de encontrar o ótimo global.



Simulated Annealing

Permite movimentos de piora temporária com probabilidade controlada por uma 'temperatura' que decresce ao longo do tempo — escapando de ótimos locais.



Local Beam Search

Explora k estados simultaneamente, compartilhando informação entre os candidatos e cobrindo regiões distintas do espaço de busca.



Algoritmos Genéticos

Mantém uma população de soluções e gera novas soluções por seleção, cruzamento e mutação — inspirado na evolução biológica darwiniana

De uma Solução para um Conjunto de Soluções

Em vez de explorar um único estado por vez, o Local Beam Search mantém k estados candidatos simultaneamente — aumentando a capacidade de exploração do espaço de busca.



Manutenção de k Estados

k estados candidatos são mantidos simultaneamente em memória, em vez de um único estado como no Hill-Climbing.



Expansão Paralela

Cada um dos k estados gera seus sucessores diretos — produzindo uma população de candidatos para avaliação.



Objetivo

Aumentar a capacidade de exploração mantendo múltiplas alternativas ativas durante a busca — reduzindo o risco de convergência prematura para ótimos locais.



Avaliação e Seleção

Todos os sucessores gerados são avaliados pela função de fitness. Os k melhores de toda a população são selecionados para a próxima iteração.

Inicialização com $k = 3$ Estados

Cada estado representa uma configuração das 9 rainhas como um vetor $S = [c_1, c_2, \dots, c_9]$, onde c_i é a coluna da rainha na linha i .

Estado	Configuração	Conflitos $h(s)$
S_1	[1, 5, 8, 6, 3, 7, 2, 9, 4]	8
S_2	[2, 7, 4, 1, 9, 6, 8, 3, 5]	6
S_3	[4, 2, 7, 3, 6, 9, 1, 8, 5]	7



Expansão

Cada um dos 3 estados gera seus 72 vizinhos (9 rainhas $\times$ 8 colunas alternativas), totalizando 216 estados candidatos.



Avaliação

A função $h(s)$ é calculada para todos os 216 candidatos – contando os pares de rainhas em conflito em cada configuração.



Seleção dos k Melhores

Os 3 estados com menor $h(s)$ de toda a população de 216 candidatos são selecionados para a próxima iteração – independentemente de qual estado os gerou.

Diferença do Hill-Climbing: os 3 estados evoluem em conjunto e compartilham informação – um estado pode "herdar" o melhor vizinho de outro estado.

Cada Estado Gera seus Sucessores

Um movimento é definido como mover uma rainha para outra coluna na mesma linha. Cada estado gera 72 sucessores (9 rainhas $\times$ 8 colunas alternativas).



Sucessores de $S_1 = [1, 5, 8, 6, 3, 7, 2, 9, 4]$

$S_{1a} = [2, 5, 8, 6, 3, 7, 2, 9, 4] \rightarrow 7$ conflitos

$S_{1b} = [1, 6, 8, 6, 3, 7, 2, 9, 4] \rightarrow 6$ conflitos

$S_{1c} = [1, 5, 9, 6, 3, 7, 2, 9, 4] \rightarrow 9$ conflitos

... (72 vizinhos no total)



Sucessores de $S_2 = [2, 7, 4, 1, 9, 6, 8, 3, 5]$

$S_{2a} = [1, 7, 4, 1, 9, 6, 8, 3, 5] \rightarrow 5$ conflitos

$S_{2b} = [2, 8, 4, 1, 9, 6, 8, 3, 5] \rightarrow 6$ conflitos

$S_{2c} = [2, 7, 3, 1, 9, 6, 8, 3, 5] \rightarrow 4$ conflitos ←
melhor

... (72 vizinhos no total)



Sucessores de $S_3 = [4, 2, 7, 3, 6, 9, 1, 8, 5]$

$S_{3a} = [3, 2, 7, 3, 6, 9, 1, 8, 5] \rightarrow 6$ conflitos

$S_{3b} = [4, 1, 7, 3, 6, 9, 1, 8, 5] \rightarrow 7$ conflitos

$S_{3c} = [4, 2, 6, 3, 6, 9, 1, 8, 5] \rightarrow 5$ conflitos

... (72 vizinhos no total)

Todos os sucessores dos 3 estados são reunidos em um único pool de 216 candidatos. O algoritmo seleciona os 3 com menor $h(s)$ – independentemente de qual estado pai os gerou.



Total de candidatos avaliados: 3 estados $\times$ 72 vizinhos = 216 configurações por iteração.

Avaliando Todos os Sucessores

Após gerar os sucessores de cada estado, calculamos $h(s)$ para todos os candidatos e reunimos em um único pool para seleção.

Estado	Conflitos $h(s)$
S_1a	7
S_1b	$4 \leftarrow \text{top 3}$
S_1c	9
S_2a	$3 \leftarrow \text{melhor}$
S_2b	6
S_2c	5
S_3a	8
S_3b	$4 \leftarrow \text{top 3}$
S_3c	10

O algoritmo seleciona os 3 estados com menor $h(s)$ do pool completo: **S_2a ($h=3$)**, **S_1b ($h=4$)** e **S_3b ($h=4$)**. Estes tornam-se os novos estados da próxima iteração.



Nova Geração Selecionada

$S_1' = S_2a \rightarrow h = 3$

$S_2' = S_1b \rightarrow h = 4$

$S_3' = S_3b \rightarrow h = 4$



Melhoria em Uma Iteração

Geração anterior: h médio = 7,0
(valores: 8, 6, 7)

Nova geração: h médio = 3,7 (valores: 3, 4, 4)

Melhoria de **47%** em uma única iteração.

A Conexão com os Algoritmos Genéticos

O Local Beam Search introduz o conceito de população e seleção. Mas e se as soluções também pudessem gerar descendentes umas das outras?



Local Beam Search

Os k melhores candidatos são simplesmente selecionados da população de sucessores. Não há troca de informação entre os estados — cada um evolui de forma independente.



Algoritmo Genético

Os candidatos selecionados também participam da criação de novas soluções — combinando material genético entre si via cruzamento e introduzindo variação via mutação.

A Teoria da Seleção Natural de Darwin

Os Algoritmos Genéticos (AGs) transpõem a teoria darwiniana de evolução de populações biológicas para o ambiente digital — mapeando conceitos biológicos em operadores computacionais.



Princípio de Sobrevivência

Indivíduos com melhor adaptação ao ambiente (**melhor fitness**) herdam maiores chances de repasse de **genes** — os mais aptos sobrevivem e se reproduzem.



Variabilidade Genética

Mutações aleatórias introduzem variabilidade genética e evitam a homogeneidade populacional precoce — essencial para escapar de ótimos locais.



Evolução por Gerações

A cada geração, a população torna-se progressivamente mais adaptada ao ambiente — convergindo para soluções de alta qualidade ao longo do tempo.

"Na natureza, a evolução não projeta soluções — ela as descobre por seleção, recombinação e variação ao longo de gerações."

Da Seleção Natural à Otimização Computacional

Os Algoritmos Genéticos simulam o processo evolutivo darwiniano — cada conceito biológico possui um análogo computacional preciso.

1) População de Indivíduos → População de Soluções

Um conjunto de organismos vivos corresponde a um conjunto de soluções candidatas para o problema de otimização.

2) Avaliação → Fitness

A adaptação de cada indivíduo ao ambiente é medida pela **função de fitness** — que quantifica a qualidade de cada solução candidata.

3) Seleção → Seleção

Indivíduos mais adaptados possuem maior probabilidade de contribuir para a próxima geração — favorecendo soluções de maior qualidade.

4) Reprodução → Cruzamento

Características de diferentes indivíduos são combinadas para produzir descendentes — novas soluções que herdam partes de seus pais.

5) Mutação → Mutação

Pequenas alterações aleatórias introduzem diversidade genética — explorando novas regiões do espaço de busca e evitando convergência prematura.

6) Nova Geração → Nova Geração

Ao longo das gerações, a população torna-se progressivamente mais adaptada — emergindo soluções de qualidade crescente.

"Um Algoritmo Genético não busca diretamente a solução ótima — ele simula um processo de evolução para que soluções cada vez melhores emergjam ao longo das gerações."

A Grande Diferença entre Beam Search e AGs

A distinção fundamental entre o Stochastic Beam Search e os Algoritmos Genéticos reside no mecanismo de reprodução — assexuada versus sexuada.



Stochastic Beam Search — Reprodução Assexuada

Cada novo estado descendente é gerado individualmente a partir de um único estado pai, com adição de ruído local (perturbação aleatória). A informação genética não é compartilhada entre indivíduos distintos.



Algoritmo Genético — Reprodução Sexuada

Cada sucessor resulta da recombinação estrutural (crossover) de características genéticas extraídas de dois pais distintos. A troca de material genético entre indivíduos é o motor da diversidade e da exploração.

Característica	Stochastic Beam Search	Algoritmo Genético
Tipo de reprodução	Assexuada	Sexuada
Número de pais	1	2
Operador principal	Perturbação (mutação)	Cruzamento (crossover)
Troca de informação	Não	Sim
Diversidade gerada	Local	Global

"O cruzamento entre dois pais permite que o AG combine blocos de boas soluções de regiões distintas do espaço de busca — algo impossível na reprodução assexuada."

Analogia Biológica vs. Implementação Computacional

Os termos biológicos são uma metáfora computacional inspirada na evolução — não uma equivalência perfeita. O AG não possui literalmente cromossomos ou DNA.

Biologia	Algoritmo Genético
Cromossomo	Representação completa de uma solução
Gene	Um componente/variável da solução
Alelo	Valor assumido por um gene
Indivíduo	Uma solução candidata
População	Conjunto de soluções candidatas
Fitness	Medida de qualidade/adaptação da solução



A Analogia Não é Perfeita

Indivíduo ↔ solução. Cromossomo ↔ representação da solução. Gene ↔ componente da solução. Esses termos são metáforas computacionais — não estruturas biológicas reais.



Tipos de Representação

Uma solução pode ser codificada como: bits (binário), inteiros, números reais, permutações ou estruturas específicas do problema.



Próximo Passo

Como representar uma solução como um cromossomo? A escolha da representação é a primeira decisão de projeto de um Algoritmo Genético.

Aplicando a Analogia: O Problema das 9 Rainhas

Para as 9 rainhas, cada indivíduo é um vetor de 9 inteiros. Cada posição é um gene; o valor nessa posição é o alelo.

"Cromossomo: [2, 5, 8, 1, 4, 7, 3, 6, 9]
Gene $g_3 = 8 \rightarrow$ o gene 3 possui o alelo 8 (rainha da linha 3 está na coluna 8)"

Indivíduo	Cromossomo	Fitness $h(s)$
Ind. 1	[2, 5, 8, 1, 4, 7, 3, 6, 9]	3 conflitos
Ind. 2	[1, 7, 4, 9, 2, 6, 8, 3, 5]	5 conflitos
Ind. 3	[4, 2, 7, 3, 6, 9, 1, 8, 5]	4 conflitos



Cromossomo

[2, 5, 8, 1, 4, 7, 3, 6, 9] – representação completa de uma configuração das 9 rainhas no tabuleiro.



Gene e Alelo

Cada gene g_i representa a linha i . O alelo é o valor da coluna. Exemplo: $g_3 = 8$ significa que a rainha da linha 3 está na coluna 8.

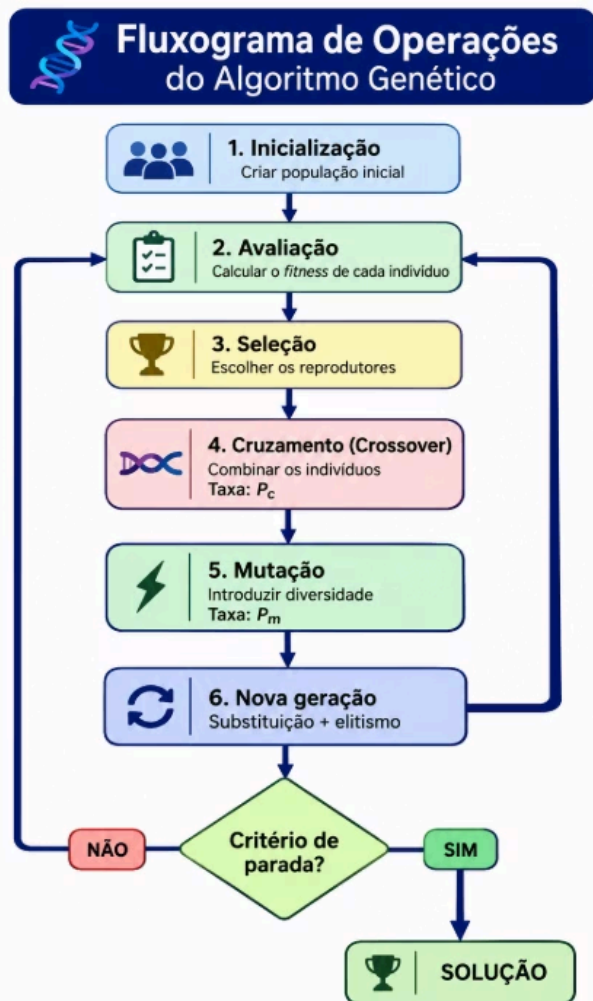


Fitness

$h(s)$ = número de pares de rainhas em conflito. Quanto menor $h(s)$, maior a qualidade do indivíduo — $h = 0$ é a solução perfeita.

Fluxograma de Operações do Algoritmo Genético

O Algoritmo Genético opera em ciclos geracionais — cada iteração aplica seleção, cruzamento e mutação sobre a população até que o critério de parada seja satisfeito.



1. Inicialização

Criar a população inicial com indivíduos gerados aleatoriamente.



2. Avaliação

Calcular o fitness de cada indivíduo da população.



3. Seleção

Escolher os indivíduos reprodutores com base no fitness — melhor fitness, maior probabilidade de seleção.



4. Cruzamento (Crossover)

Combinar pares de indivíduos para gerar descendentes. Taxa de cruzamento: P_x .



5. Mutação

Introduzir variações aleatórias nos descendentes para manter diversidade. Taxa de mutação: P_m .



6. Nova Geração

Substituir a população antiga pela nova geração (com elitismo opcional) e repetir até o critério de parada.

Principais Parâmetros do AG

A configuração correta dos parâmetros é fundamental para o desempenho do Algoritmo Genético — cada parâmetro controla um aspecto diferente do equilíbrio entre exploração e convergência.

Parâmetro	Nome	Papel no Algoritmo
N	Tamanho da população	Controla a diversidade inicial e o custo computacional por geração. N grande → mais diversidade, mais lento.
P _o	Taxa de crossover	Probabilidade de dois pais realizarem crossover. Valores típicos: 0.6 a 0.9.
P _m	Taxa de mutação	Probabilidade de um gene ser mutado. Valores típicos: 0.001 a 0.05. Muito alta → busca aleatória.
k	Tamanho do torneio	Número de indivíduos sorteados por torneio. k maior → pressão seletiva maior.
σ	Intensidade da mutação gaussiana	Desvio padrão da perturbação em representação real. σ grande → exploração. σ pequeno → refinamento.
E	Elitismo	Número de indivíduos preservados diretamente na próxima geração. Garante que a melhor solução nunca seja perdida.



Exploração × Exploração

N grande, P_m alta e k pequeno favorecem **exploração**. N pequeno, P_m baixa e k grande favorecem **exploração** (**convergência**).



Convergência Prematura

P_m muito baixa ou k muito grande podem fazer a população convergir antes de encontrar o ótimo global — perdendo diversidade cedo demais.



Ajuste Empírico

Não existe configuração universal. Os parâmetros devem ser ajustados para cada problema — frequentemente via experimentação ou meta-otimização.

Valores de referência comuns: N ∈ [50, 500] · P_o ∈ [0.6, 0.9] · P_m ∈ [0.001, 0.05] · k ∈ [2, 5] · E ∈ [1, 5% de N]

Inicialização: Criação da População Inicial

O AG não começa com uma solução – começa com várias soluções candidatas. Uma população diversificada favorece a exploração ampla do espaço de busca.



Geração Aleatória

Na estratégia mais simples, os indivíduos são gerados aleatoriamente – garantindo diversidade inicial e cobertura ampla do espaço de busca.



Diversidade Inicial

Uma população diversificada inicialmente favorece a exploração do espaço de busca – reduzindo o risco de convergência prematura para ótimos locais.



Estratégias Heurísticas

Também podem ser incorporadas soluções previamente conhecidas ou geradas por heurísticas – acelerando a convergência sem sacrificar diversidade.

Exemplo – 9 Rainhas (população inicial):

$S_1 = [2, 5, 8, 1, 4, 7, 3, 6, 9]$

$S_2 = [7, 3, 1, 9, 6, 4, 2, 8, 5]$

$S_3 = [4, 9, 2, 6, 1, 8, 5, 3, 7]$

...

Avaliação da População — Fitness

O fitness quantifica a qualidade de cada solução candidata em relação ao objetivo do problema — determinando quais indivíduos terão maior influência nas próximas gerações.



Função de Fitness

Cada indivíduo é avaliado por uma função $f(S)$ que mede sua qualidade. Para as 9 rainhas: $f(S)$ = número de pares de rainhas em conflito — quanto menor, melhor.



Critério de Qualidade

O fitness transforma o problema de otimização em um critério de seleção — indivíduos com melhor fitness terão maior probabilidade de contribuir para a próxima geração.



Melhor ≠ Sempre Escolhido

O fitness indica quais soluções são melhores, mas não garante que as melhores serão sempre selecionadas — a seleção é probabilística, preservando diversidade.

Indivíduo	Cromossomo	Conflitos $f(S)$
S_1	[2, 5, 8, 1, 4, 7, 3, 6, 9]	8
S_2	[7, 3, 1, 9, 6, 4, 2, 8, 5]	5
S_3	[4, 9, 2, 6, 1, 8, 5, 3, 7]	11
S_4	[3, 6, 9, 2, 5, 8, 1, 4, 7]	3 ← melhor

O fitness diz ao algoritmo quais soluções são melhores — mas ainda não significa que as melhores serão sempre escolhidas. Isso prepara a seleção probabilística.

Fitness: Maximização × Minimização

O problema pode ser formulado como maximização de fitness ou minimização de erro — a escolha depende da representação adotada.

Maximização

Quanto maior o fitness → melhor a solução. O AG favorece indivíduos com maior $f(s)$.
Exemplo: **maximizar lucro, precisão, cobertura.**

Minimização

Quanto menor o erro → melhor a solução. O AG favorece indivíduos com menor $h(s)$.
Exemplo: **minimizar conflitos, custo, distância.**

Exemplo: Problema das 9 Rainhas



Função de Custo $h(s)$

$h(s)$ = número de conflitos entre rainhas. $h(s) = 0$ → solução perfeita (nenhuma rainha ataca outra).



Conversão para Maximização

Para usar seleção por roleta ou ranking (que assumem maximização): $F(s) = 1 / (1 + h(s))$. Quanto menor o conflito, maior o fitness $F(s)$.

Tabela de Conversão $h(s) \rightarrow F(s)$

Estado	$h(s)$ – Conflitos	$F(s) = 1/(1+h(s))$
A	0	1,000
B	1	0,500
C	3	0,250
D	5	0,167
E	10	0,091

O estado A ($h=0$) é a solução perfeita — e recebe o maior fitness $F(s) = 1,000$. O estado E ($h=10$) tem muitos conflitos — e recebe o menor fitness $F(s) = 0,091$.

Seleção dos Reprodutores

Seleção não significa simplesmente escolher os melhores — significa aumentar a probabilidade de que boas características sejam reproduzidas.



Seleção Probabilística

Indivíduos mais adaptados recebem maior probabilidade de reprodução — mas indivíduos menos adaptados também podem ser selecionados, preservando diversidade.



Pressão Seletiva

O mecanismo de seleção controla a pressão seletiva da evolução — alta pressão converge rápido mas pode perder diversidade; baixa pressão explora mais amplamente.



Seleção por Roleta

Cada indivíduo recebe uma fatia da roleta proporcional ao seu fitness. Indivíduos com maior fitness têm maior chance de ser sorteados.



Torneio e Ranking

Torneio: k indivíduos são sorteados e o melhor vence. Ranking: indivíduos são ordenados por fitness e a probabilidade é proporcional à posição no ranking.

A seleção é o motor da evolução — ela direciona a busca para regiões promissoras do espaço de soluções sem abandonar completamente a exploração.



Os três métodos principais serão detalhados a seguir: Roleta (fitness proporcional), Torneio e Ranking.

Roleta — Probabilidade Proporcional ao Fitness

Na seleção por roleta, cada indivíduo recebe uma fatia proporcional ao seu fitness — quanto maior o fitness, maior a chance de ser selecionado. Mas nenhum indivíduo é garantido.

Indivíduo	Fitness	Probabilidade P(x)
A	10	10/26 = 38,5%
B	7	7/26 = 26,9%
C	5	5/26 = 19,2%
D	3	3/26 = 11,5%
E	1	1/26 = 3,8%
Total	26	100%

$$P(x_i) = f(x_i) / \sum f(x_j) \rightarrow \text{Fitness total} = 10 + 7 + 5 + 3 + 1 = 26$$

- Na roleta, você não escolhe simplesmente o indivíduo com maior probabilidade — você realiza um **sorteio aleatório ponderado pelo fitness**.
- Exemplo: gera-se um número aleatório $r \in [0, 1]$. Se $r \leq 0,385 \rightarrow$ seleciona **A**. Se $0,385 < r \leq 0,654 \rightarrow$ seleciona **B**. Se $0,654 < r \leq 0,846 \rightarrow$ seleciona **C**. E assim por diante.
- Isso significa que **A pode não ser escolhido**, e **E pode ser** — com probabilidade 3,8%.
- Não existe uma regra, mas normalmente se fazem **N** sorteios.

Roleta — Realizando os Sorteios

Para selecionar N pais, realizamos N sorteios independentes — cada sorteio gera um número aleatório $r \in [0, 1]$ e seleciona o indivíduo cujo intervalo acumulado contém r .

Sorteio	r gerado	Intervalo	Selecionado
1	0,62	$0,385 < r \leq 0,654$	B
2	0,21	$r \leq 0,385$	A
3	0,81	$0,654 < r \leq 0,846$	C
4	0,37	$r \leq 0,385$	A

Pais selecionados: B, A, C, A — A apareceu duas vezes. Isso é perfeitamente possível: indivíduos com maior fitness podem ser selecionados múltiplas vezes no mesmo ciclo reprodutivo.



Seleção com Reposição

Cada sorteio é independente — o mesmo indivíduo pode ser selecionado múltiplas vezes. Isso é chamado de seleção com reposição.



Limitação da Roleta

Se um indivíduo tiver fitness muito superior, ele dominará os sorteios — reduzindo a diversidade. Métodos como Torneio e Ranking são mais robustos a essa limitação.

Ranking — Seleção pela Posição

No ranking, o valor absoluto do fitness deixa de ser o elemento principal — o que importa é a posição relativa do indivíduo na população ordenada.

Posição	Indivíduo	Fitness	Rank	$P(i) = \text{Rank}_i / 15$
1º	A	14	5	33,3% (5/15)
2º	B	7	4	26,7% (4/15)
3º	C	5	3	20,0% (3/15)
4º	D	3	2	13,3% (2/15)
5º	E	1	1	6,7% (1/15)

- Assumindo que maior fitness = melhor. A soma dos ranks é $5+4+3+2+1=15$. Portanto: $P(i) = \text{Rank}_i / \Sigma \text{Rank}$. O 1º lugar (maior fitness) recebe o rank mais alto — a probabilidade é proporcional à posição no ranking, não ao valor absoluto do fitness.
- A probabilidade é proporcional à posição no ranking — não ao valor absoluto do fitness. O 1º lugar sempre recebe a maior fatia, independentemente de quão superior seja seu fitness.



Problema da Roleta com Fitness Extremo

Se os fitness forem: A=1000, B=100, C=10, D=5, E=1 → total=1116

$P(A) = 89,6\%$ — A domina completamente os sorteios, eliminando a diversidade.



Vantagem do Ranking

Com os mesmos indivíduos, o ranking mantém: 1º A, 2º B, 3º C, 4º D, 5º E.

A diferença relativa de fitness deixa de dominar — todos os indivíduos mantêm chances razoáveis de reprodução.

 O ranking é especialmente útil quando os valores de fitness variam em ordens de magnitude — evitando que um único indivíduo domine toda a seleção.

Torneio — Competição entre Candidatos

Não comparamos toda a população — sorteamos um pequeno grupo de ***k* indivíduos** e o melhor do grupo vence. O processo se repete até selecionar todos os pais necessários. **Selecionamos *k* indivíduos *N* vezes necessárias.**



Torneio 1 (k = 3)

Sorteados: A (fitness=10), C (fitness=5), D (fitness=3)
Vencedor: **A** — maior fitness do grupo.



Torneio 2 (k = 3)

Sorteados: B (fitness=7), C (fitness=5), E (fitness=1)
Vencedor: **B** — maior fitness do grupo.

O vencedor de cada torneio é selecionado como pai — independentemente do fitness dos demais indivíduos da população.

Tamanho do Torneio (k)	Pressão Seletiva	Diversidade
k = 2	Baixa	Alta
k = 3 a 5	Moderada	Moderada
k = N (população toda)	Máxima	Baixa



Vantagem do Torneio

Não requer o cálculo do fitness total da população — mais eficiente computacionalmente.
Robusto a escalas de fitness muito diferentes entre indivíduos.



Parâmetro k

k controla a pressão seletiva: k grande → os melhores dominam (convergência rápida, menos diversidade). k pequeno → mais diversidade, exploração mais ampla.

Comparando os Métodos de Seleção

Cada método define de forma diferente quem tem maior chance de se reproduzir.

Método	O que determina a seleção?	Pressão Seletiva
Roleta	Fitness absoluto	Depende da distribuição dos fitness
Torneio	Melhor dentro de um grupo sorteado	Controlada pelo parâmetro k
Ranking	Posição relativa na população ordenada	Mais estável e previsível

🤔 Qual método selecionaria o indivíduo A com maior probabilidade — se A tiver fitness muito superior aos demais?



Roleta

$P(i) = f_i / \sum_j f_j$ — probabilidade proporcional ao fitness absoluto.



Torneio

$\text{pai} = \operatorname{argmax} f_i, i \in T$ — o melhor indivíduo dentro do grupo sorteado T vence.



Ranking

$P(i) = f(\text{posição}_i)$ — probabilidade proporcional à posição no ranking, não ao fitness absoluto.

📌 A **Roleta** tende a selecionar A com maior probabilidade quando seu fitness é muito superior — pois $P(A)$ cresce proporcionalmente. O **Torneio** e o **Ranking** distribuem melhor as chances entre os demais.

Cruzamento — Combinação de Soluções

Dois indivíduos selecionados atuam como pais. O crossover combina partes de seus cromossomos para gerar novos indivíduos — recombinaando características promissoras encontradas em diferentes soluções.



Recombinação Genética

Partes do cromossomo de cada pai são combinadas para formar um filho com características de ambos.



Ponto de Corte

Um ponto de corte divide o cromossomo. O filho herda os genes anteriores ao corte do Pai 1 e os posteriores do Pai 2.



Probabilidade P_o

O cruzamento só ocorre com probabilidade P_o — caso contrário, os filhos são cópias diretas dos pais.

Exemplo: Crossover de Ponto Único

Indivíduo	Cromossomo	Descrição
Pai 1	2 5 8 1 4 7 3 6 9	Genes antes do corte (posição 3)
Pai 2	7 3 1 9 6 4 2 8 5	Genes após o corte (posição 3)
↓ crossover ↓		
Filho	2 5 8 9 6 4 2 8 5	Herda início do Pai 1 + fim do Pai 2

Probabilidade de Crossover — P_o

Nem todo par de pais realiza crossover — a probabilidade P_o controla com que frequência o operador é aplicado. Se não houver crossover, os filhos são cópias diretas dos pais.

01	02	03
Seleção dos Pais P_1 e P_2 são selecionados da população pelo operador de seleção (roleta, torneio ou ranking).	Sorteio $r \sim U(0,1)$ Um número aleatório r é sorteado de uma distribuição uniforme entre 0 e 1.	Comparação $r < P_o$ Se $r < P_o \rightarrow$ realiza crossover. Se $r \geq P_o \rightarrow$ os filhos são cópias dos pais (sem recombinação).
04	05	
Crossover Aplica o operador escolhido: BLX- α , aritmético, ponto único, etc. sobre os genes (x_{1i}, x_{2i}) .	Filhos Gerados Os filhos resultantes entram na nova população.	

Exemplo: $P_o = 0.8$

Elemento	Valor
Pai 1 (P_1)	[2.0, 4.5, 8.0]
Pai 2 (P_2)	[6.0, 3.0, 5.0]
Probabilidade de crossover (P_o)	0.8
Sorteio (r)	0.65
Condição ($r < P_o$?)	$0.65 < 0.8 \rightarrow$  Realiza crossover

 $r = 0.65 < P_o = 0.8 \rightarrow$ crossover realizado!

 Valores típicos: $P_o \in [0.6, 0.9]$. Com $P_o = 0.8$, cerca de 80% dos pares realizam crossover e 20% passam seus genes sem recombinação — preservando soluções já boas na população.

Crossover de Ponto Único

O tipo mais simples de crossover — um ponto de corte aleatório divide o cromossomo em dois segmentos, que são trocados entre os pais para gerar dois filhos.



Ponto Aleatório

Um ponto de corte é sorteado aleatoriamente ao longo do cromossomo — dividindo-o em segmento esquerdo e segmento direito.



Troca de Segmentos

Filho 1 herda o segmento esquerdo do Pai 1 e o segmento direito do Pai 2. Filho 2 faz o inverso.



Dois Filhos

Cada par de pais gera dois filhos — garantindo que nenhum material genético seja desperdiçado.

Exemplo: Cromossomos

Indivíduo	Cromossomo	Origem dos Genes
Pai 1	2 5 8 1 4 7 3 6 9	Segmento esquerdo (antes do corte)
Pai 2	7 3 1 9 6 4 2 8 5	Segmento direito (após o corte)
↑ ponto de corte sorteado após a posição 3 ↑		
Filho 1	2 5 8 9 6 4 2 8 5	Esq. do Pai 1 + Dir. do Pai 2
Filho 2	7 3 1 1 4 7 3 6 9	Esq. do Pai 2 + Dir. do Pai 1

Crossover de Múltiplos Pontos

Em vez de um único corte, utilizamos dois ou mais pontos de corte – alternando os segmentos entre os pais para produzir uma mistura maior de características.



Múltiplos Cortes

Dois ou mais pontos de corte são sorteados, dividindo o cromossomo em três ou mais segmentos alternados.



Alternância de Segmentos

Os segmentos são alternados entre os pais: Filho 1 herda os segmentos ímpares do Pai 1 e os pares do Pai 2. Filho 2 faz o inverso.



Menos Viés Posicional

Com mais pontos de corte, reduz-se o viés posicional do crossover de ponto único – genes distantes têm maior chance de serem recombinados juntos.

Exemplo: Dois Pontos de Corte

Indivíduo	Cromossomo	Descrição
Pai 1	2 5 8 1 4 7 3 6 9	Segmentos 1 e 3 (posições 1-3 e 7-9)
Pai 2	7 3 1 9 6 4 2 8 5	Segmento 2 (posições 4-6)
↑ cortes após as posições 3 e 6 – três segmentos alternados ↑		
Filho 1	2 5 8 9 6 4 3 6 9	Seg.1 do Pai 1 + Seg.2 do Pai 2 + Seg.3 do Pai 1
Filho 2	7 3 1 1 4 7 2 8 5	Seg.1 do Pai 2 + Seg.2 do Pai 1 + Seg.3 do Pai 2

Com mais pontos de corte, a recombinação é mais rica – genes de regiões distantes do cromossomo têm maior chance de serem combinados. Isso reduz o viés posicional presente no crossover de ponto único.

Crossover Uniforme

Sem pontos de corte fixos – cada gene é decidido independentemente por uma máscara aleatória, produzindo uma mistura muito mais granular dos pais.



Máscara Aleatória

Uma **máscara binária** é **gerada aleatoriamente** – cada posição indica qual pai fornecerá o gene correspondente (1 = Pai 1, 0 = Pai 2).



Decisão Gene a Gene

Cada gene é escolhido de forma independente – não há segmentos contíguos. Qualquer combinação de genes dos dois pais é possível.



Mistura Granular

O crossover uniforme produz a maior diversidade de recombinação possível – ideal quando não há dependência posicional entre os genes.

Exemplo: Máscara

Indivíduo	Cromossomo	Descrição
Pai 1	2 5 8 1 4 7 3 6 9	Fornece o gene onde a máscara = 1
Pai 2	7 3 1 9 6 4 2 8 5	Fornece o gene onde a máscara = 0
Máscara	1 0 1 0 0 1 1 0 1	1 → Pai 1 · 0 → Pai 2

↓ aplicando a máscara gene a gene ↓

Filho	2 3 8 9 6 7 3 8 9	Cada gene selecionado individualmente pela máscara
-------	-------------------	--

Crossover BLX- α — Representação Real

Para cromossomos com variáveis contínuas, o BLX- α permite explorar uma região maior do que o simples intervalo entre os pais — estendendo o espaço de busca por $\alpha \cdot d$ em cada extremidade.



Distância entre os Pais

A distância $d = x_2 - x_1$ define o intervalo base. O BLX- α estende esse intervalo por $\alpha \cdot d$ em cada extremidade (x_1 e x_2 são alelos de um mesmo gene, de dois pais diferentes).



Intervalo Estendido

O filho é gerado aleatoriamente dentro de $[x_1 - \alpha \cdot d, x_2 + \alpha \cdot d]$ — podendo assumir valores fora do intervalo dos pais.



Exploração Além dos Pais

Com $\alpha > 0$, o algoritmo não fica limitado aos valores dos pais — ele explora uma região ao redor deles, aumentando a diversidade.

Exemplo: $\alpha = 0.5$

Elemento	Valor
Pai 1 (x_1)	2.31
Pai 2 (x_2)	1.87
Distância ($d = x_1 - x_2 $)	0.44
α	0.5
Intervalo do filho	$[1.87 - 0.5 \times 0.44, 2.31 + 0.5 \times 0.44] = [1.65, 2.53]$

i O intervalo $[1.65, 2.53]$ é maior que o intervalo original $[1.87, 2.31]$ entre os pais. Com $\alpha = 0.5$, o filho pode assumir qualquer valor real dentro desse intervalo estendido. O mesmo cálculo é aplicado a cada um dos 4 genes: $[2.31, 0.87, 14.52, 3.76]$ e $[1.87, 0.25, 13.00, 3.00]$.

Crossover Aritmético — Interpolação Linear

Uma combinação linear ponderada dos pais — o filho é gerado como uma interpolação entre os dois valores, controlada pelo parâmetro λ .



Combinação Linear

Filho 1 = $\lambda \cdot x_1 + (1-\lambda) \cdot x_2$ · Filho 2 = $(1-\lambda) \cdot x_1 + \lambda \cdot x_2$. Os dois filhos são complementares — somam sempre os mesmos pesos.



Parâmetro $\lambda \in [0, 1]$

λ controla o peso de cada pai. $\lambda = 0.5$ gera a média aritmética. λ próximo de 0 ou 1 gera um filho mais parecido com um dos pais.



Sempre dentro do Intervalo

Diferente do BLX- α , o crossover aritmético nunca gera valores fora do intervalo $[x_1, x_2]$ — é uma interpolação pura, sem exploração além dos pais.

Exemplo: $\lambda = 0.3$

Indivíduo	Cálculo	Valor do Alelo no Gene
Pai 1 (x_1)	—	2
Pai 2 (x_2)	—	8
Filho 1	$0.3 \times 2 + 0.7 \times 8$	6.2
Filho 2	$0.7 \times 2 + 0.3 \times 8$	3.8

Pai 1 = 2 · Pai 2 = 8 · ↓ crossover aritmético ($\lambda = 0.3$) ↓ · Filho 1 = 6.2 · Filho 2 = 3.8

i O crossover aritmético é ideal para cromossomos de representação contínua. Os dois filhos são sempre complementares: os pesos de λ e $(1-\lambda)$ se invertem entre Filho 1 e Filho 2 — garantindo que o material genético de ambos os pais seja preservado.

Mutação — Preservando a Diversidade

Após o crossover, os descendentes podem sofrer pequenas alterações aleatórias — introduzindo material genético que pode não estar presente na população atual.



Variação Aleatória

Um ou mais genes são alterados aleatoriamente com probabilidade P_m — **normalmente muito baixa (0.1% a 5%) para não destruir boas soluções.**



Preserva a Diversidade

Evita que a população convirja prematuramente para um ótimo local — garantindo que novas regiões do espaço de busca possam ser exploradas.



Complementa o Crossover

Crossover combina o que já existe na população. Mutação introduz variação genuinamente nova — os dois operadores são complementares.

Exemplo: Mutação de um Gene

Momento	Cromossomo	Observação
Antes	2 5 8 1 4 7 3 6 9	Cromossomo original após o crossover
Depois	2 5 8 1 4 7 3 8 9	Gene na posição 8 alterado de 6 → 8



Crossover combina o que já existe — Mutação introduz variação genuinamente nova.



P_m deve ser baixa o suficiente para não transformar o AG em uma busca aleatória, mas alta o suficiente para manter diversidade. Valores típicos: $P_m \in [0.001, 0.05]$ para representação binária.

Mutação Bit-Flip — Representação Binária

A forma mais simples de mutação — cada gene binário pode ser invertido com probabilidade P_m . O operador é extremamente simples: 0 vira 1, e 1 vira 0.



Inversão de Bit

Cada gene tem exatamente dois estados possíveis: 0 e 1. A mutação bit-flip simplesmente inverte o valor — 0 → 1 ou 1 → 0.



Sorteio Gene a Gene

Para cada gene, sorteamos um número aleatório $r \in [0,1]$. Se $r < P_m$, o bit é invertido. Caso contrário, o gene permanece inalterado.



Exclusivo para Binários

O bit-flip é específico para cromossomos binários. Para representações inteiras ou reais, outros operadores de mutação são mais adequados.

Exemplo: Inversão do Gene na Posição 3

Momento	Cromossomo	Observação
Original	1 0 1 1 0 0 1 0	Cromossomo antes da mutação
Mutação	1 0 0 1 0 0 1 0	Gene 3: bit 1 → 0 (invertido)

↑ posição 3 selecionada para mutação: 1 → 0

i Em cromossomos binários, o bit-flip é o operador de mutação mais natural — cada posição tem exatamente dois estados, tornando a inversão a única transformação possível. Para representações inteiras ou reais, utilizam-se operadores como mutação gaussiana ou por permutação.

Mutação Gaussiana — Representação Real

Para cromossomos com variáveis contínuas, a mutação gaussiana adiciona uma pequena perturbação numérica a cada gene — sorteada a partir de uma distribuição normal $N(0, \sigma^2)$.



Perturbação Normal

O ruído $\epsilon \sim N(0, \sigma^2)$ é sorteado de uma distribuição normal com média 0 e desvio σ . Valores próximos de zero são mais prováveis — a mutação tende a ser pequena.



Controle via σ

σ controla a intensidade da perturbação. **σ grande → mutações mais agressivas (exploração). σ pequeno → mutações sutis (refinamento local).**



Para Valores Reais

Diferente do bit-flip, a mutação gaussiana é contínua — adequada para cromossomos com genes reais. **A fórmula é: $x'_i = x_i + \epsilon$, onde $\epsilon \sim N(0, \sigma^2)$.**

Exemplo: Perturbação de Dois Genes

Gene	Valor Original x_i	Perturbação ϵ	Valor Mutado x'_i
Gene 1	2.31	+0.12	2.43
Gene 2	0.87	0.00	0.87 (não mutado)
Gene 3	14.52	-0.35	14.17
Gene 4	3.76	0.00	3.76 (não mutado)

Antes: [2.31 0.87 14.52 3.76] → Depois: [2.43 0.87 14.17 3.76]



A mutação gaussiana é o operador padrão para cromossomos de representação real. Ao contrário do bit-flip, ela não altera o tipo do gene — apenas o perturba levemente, permitindo um refinamento contínuo das soluções ao longo das gerações.

Exemplo: Uma Geração Completa

Geração 0 – População Inicial

Indivíduo	Cromossomo	Fitness
A	1 0 1 1 0	10
B	1 1 0 0 1	8
C	0 1 1 1 0	6
D	1 0 0 1 1	5



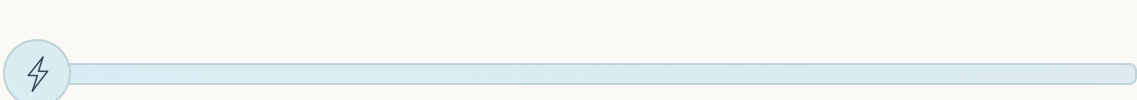
1. Seleção

Pares selecionados com base no fitness:
Par 1 → A + C · Par 2 → B + A



2. Crossover (corte após posição 3)

Par 1: A=101 | 10 × C=011 | 10 → F₁=10110 · F₂=01110
Par 2: B=110 | 01 × A=101 | 10 → F₃=11010 · F₄=10101



3. Mutação

F₃ = 1 1 0 1 0 → gene 3 invertido → F₃' = 1 1 1 1 0



4. Nova População

Geração 1: F₁ · F₂ · F₃' · F₄ — prontos para avaliação de fitness.

Resultado: Geração 0 → Geração 1

Indivíduo	Cromossomo	Origem	Fitness
F ₁	1 0 1 1 0	Esq(A) + Dir(C)	10
F ₂	0 1 1 1 0	Esq(C) + Dir(A)	6
F ₃ '	1 1 1 1 0	Esq(B) + Dir(A) + mutação	—
F ₄	1 0 1 0 1	Esq(A) + Dir(B)	—

Formação da Nova População

Após o crossover e a mutação, os novos indivíduos são incorporados à população — substituindo total ou parcialmente os indivíduos da geração anterior.



Modelo Geracional

Toda a população é substituída a cada geração — os filhos formam completamente a nova população. Simples e amplamente utilizado.



Modelo Steady-State

Apenas parte da população é substituída por geração — os piores indivíduos são removidos e os novos filhos ocupam seus lugares.



Elitismo

Os melhores indivíduos da geração atual são copiados diretamente para a próxima — garantindo que a melhor solução encontrada nunca seja perdida.

Fluxo de uma Geração



Geração t

População atual: S_1, S_2, S_3, S_4, S_5



Seleção

Indivíduos selecionados como pais com base no fitness



Crossover

Pais combinados para gerar filhos



Mutação

Pequenas perturbações aleatórias nos filhos



Geração t+1

Nova população: $S_1', S_2', S_3', S_4', S_5'$

Uma geração termina quando uma nova população é completamente formada — e o ciclo recomeça.

Evolução ao Longo das Gerações

A cada geração, a população tende a melhorar — o fitness médio sobe e o melhor indivíduo encontrado se aproxima do ótimo global.

Geração 0

População inicial aleatória. Diversidade máxima, fitness médio baixo.

Avaliação

Cada indivíduo recebe um valor de fitness $f(x)$.

Seleção

Indivíduos mais aptos têm maior probabilidade de reprodução.

Crossover

Pares de pais geram filhos com características combinadas.

Mutação

Pequenas perturbações aleatórias preservam a diversidade.

Geração $t+1$

Nova população formada. Ciclo recomeça.

O que muda a cada geração?



Maximização

- **Fitness médio** ↑ (a população melhora em média)
- **Melhor fitness** ↑ (o melhor indivíduo encontrado melhora)
- **Diversidade** ↓ (a população converge gradualmente)



Minimização

- **Erro médio** ↓ (a população erra menos em média)
- **Melhor erro** ↓ (o melhor indivíduo comete menos erro)
- **Diversidade** ↓ (a população converge gradualmente)

O AG termina quando: o número máximo de gerações N é atingido · ou · uma solução satisfatória é encontrada · ou · a população convergiu.

Exemplo: Sintonia de PID com Algoritmo Genético

O AG busca automaticamente os ganhos $[K_p, K_i, K_d]$ que produzem a melhor resposta ao degrau – sem necessidade de ajuste manual ou modelo linearizado.

A Planta

Função de transferência de segunda ordem:

$$G(s) = \frac{1}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

Com parâmetros:

- $\zeta = 0.5$ (amortecimento)
- $\omega_n = 1$ rad/s (frequência natural)

Portanto:

$$G(s) = \frac{1}{s^2 + s + 1}$$

O Controlador PID

Função de transferência do PID:

$$C(s) = K_p + \frac{K_i}{s} + K_d \cdot s$$

Ou equivalentemente:

$$C(s) = \frac{K_d s^2 + K_p s + K_i}{s}$$

O AG deve encontrar o vetor:

$$[K_p, K_i, K_d] \in \mathbb{R}^3$$

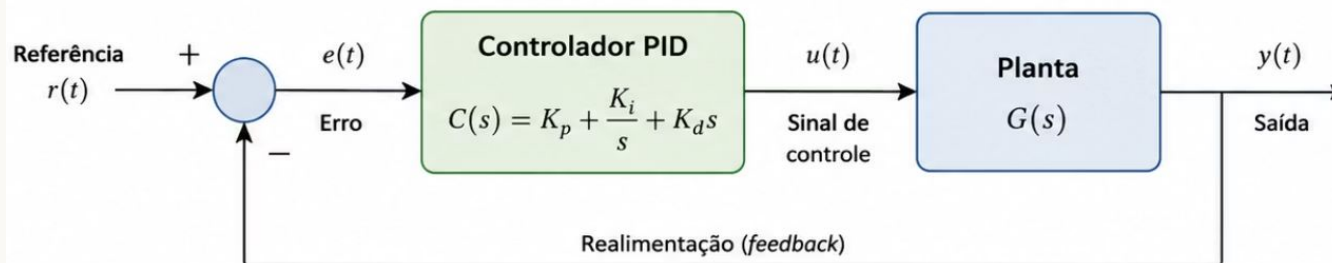
Como o AG Resolve o Problema

- i** O espaço de busca é $[K_p, K_i, K_d] \in \mathbb{R}^3$ – o AG explora esse espaço sem precisar de gradiente, sem exigir diferenciabilidade da função de custo e sem depender de um ponto inicial bem escolhido.

Diagrama de Blocos do Sistema de Controle PID

O AG otimiza os ganhos $[K_p, K_i, K_d]$ do controlador PID – avaliando a qualidade da resposta ao degrau a cada geração.

DIAGRAMA DE BLOCOS DO SISTEMA DE CONTROLE



PLANTA (EXEMPLO)

Sistema de 2ª ordem subamortecido

$$G(s) = \frac{1}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

onde:

- ω_n : frequência natural
- ζ : fator de amortecimento

CONTROLADOR PID

P
(Proporcional)

$$u_p(t) = K_p e(t)$$

Ação proporcional ao erro atual.
Reduz o tempo de subida.

I
(Integral)

$$u_i(t) = K_i \int_0^t e(\tau) d\tau$$

Ação proporcional ao erro acumulado.
Elimina o erro em regime permanente.

D
(Derivativa)

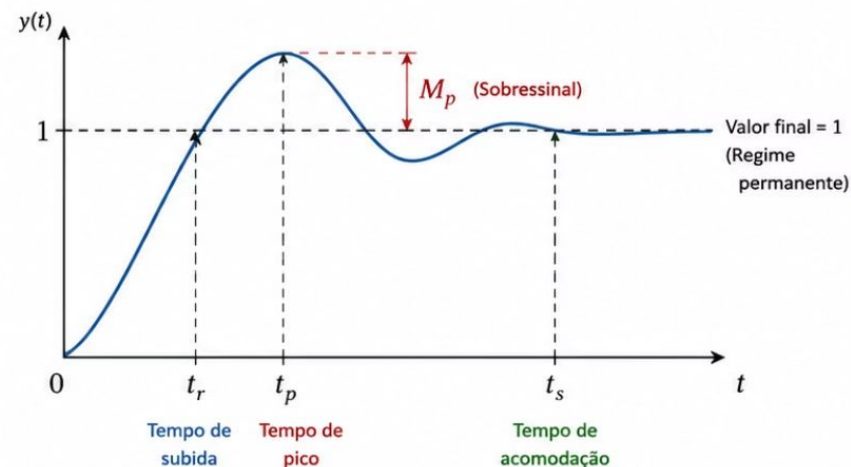
$$u_d(t) = K_d \frac{de(t)}{dt}$$

Ação proporcional à taxa de variação do erro.
Reduz o sobressinal e melhora a estabilidade.

Lei de controle (soma das ações):

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

RESPOSTA AO DEGRAU UNITÁRIO



Quem é o Indivíduo no Problema do PID?

A analogia biológica aplicada a um problema real de engenharia – cada configuração de PID é um indivíduo, e o AG evolui uma população de controladores candidatos.

Conceito Biológico	Conceito do AG	No Problema do PID
Cromossomo	Vetor de parâmetros	$[K_p, K_i, K_d] \in \mathbb{R}^3$
Gene	Parâmetro individual	$K_p \cdot \text{ou} \cdot K_i \cdot \text{ou} \cdot K_d$
Indivíduo	Solução candidata completa	Uma configuração completa do PID
População	Conjunto de soluções	Várias configurações de PID avaliadas em paralelo
Fitness	Qualidade da solução	ISE ou ITAE da resposta ao degrau (menor = melhor)

Exemplo: População Inicial

Indivíduo	K_p	K_i	K_d
Indivíduo 1	2.5	1.2	0.8
Indivíduo 2	4.1	0.7	1.3
Indivíduo 3	1.8	2.1	0.2
Indivíduo 4	3.3	0.9	0.6
...	...	...	...



Cromossomo Real

$X = [K_p, K_i, K_d]$ – vetor de 3 genes reais. Cada gene é um ganho do controlador PID. O crossover BLX- α e a mutação gaussiana são os operadores naturais para esse tipo de representação.



Fitness = Qualidade do Controle

Cada indivíduo é simulado: aplica-se um degrau e mede-se o erro. $ISE = \int e^2(t)dt$. Menor ISE $\rightarrow$ maior fitness $\rightarrow$ maior chance de reprodução.



Evolução dos Controladores

A cada geração, os melhores controladores se reproduzem – combinando e perturbando seus ganhos. Após N gerações, o melhor indivíduo é o controlador PID otimizado.

Qual será o Fitness no Problema do PID?

O fitness deve capturar a qualidade da resposta ao degrau – penalizando erro, lentidão, overshoot e oscilações excessivas.



Rapidez

A resposta deve atingir o valor desejado rapidamente – tempo de subida e tempo de acomodação pequenos.



Pouco Erro

O erro em regime permanente $e(t) = r(t) - y(t)$ deve ser mínimo – o sistema deve rastrear a referência com precisão.



Pouco Overshoot

A resposta não deve ultrapassar excessivamente o valor desejado – picos elevados indicam instabilidade ou ganhos mal ajustados.



Sem Oscilações

A resposta deve convergir suavemente – oscilações persistentes indicam margens de fase e ganho insuficientes.

Função de Custo: IAE

Definição do IAE

$$IAE = \int_0^T |e(t)| dt \quad \text{onde } e(t) = r(t) - y(t)$$

Integra o valor absoluto do erro ao longo do tempo – penaliza tanto erros positivos quanto negativos de forma igual.

Conversão para Maximização

$$\text{Fitness} = \frac{1}{1 + IAE}$$

IAE pequeno → Fitness alto → maior chance de reprodução. IAE = 0 → Fitness = 1 (solução perfeita).

Comparando Controladores

Controlador PID	IAE	Fitness = 1/(1+IAE)
A	3.2	0.238
B	1.8	0.357

Estrutura Geral do AG em Python

A estrutura conceitual do loop principal. Cada linha corresponde a um passo do ciclo evolutivo.

```
for generation in range(N_GENERATIONS):

    # Passo 2: Avaliação
    fitness = evaluate_population(population)

    # Passo 3: Seleção
    parents = selection(population, fitness)

    # Passo 4: Cruzamento
    children = crossover(parents)

    # Passo 5: Mutação
    children = mutation(children)

    # Passo 6: Nova população
    population = create_new_population(children)
```

01

evaluate_population()

Calcula o fitness de cada indivíduo. Retorna um vetor de valores $f(x)$ para toda a população.

02

selection()

Seleciona os pais com base no fitness. Pode usar roleta, torneio ou ranking.

03

crossover()

Combina pares de pais para gerar filhos. Ponto único, múltiplos pontos, uniforme, BLX- α ou aritmético.

04

mutation()

Aplica perturbações aleatórias nos filhos. Bit-flip para binários, gaussiana para reais.

05

create_new_population()

Forma a nova geração. Pode incluir elitismo, modelo geracional ou steady-state.

Saída do AG: Evolução Geração a Geração

A cada geração, o AG imprime o melhor fitness encontrado até o momento e os ganhos $[K_p, K_i, K_d]$ do melhor controlador – mostrando a convergência progressiva.

Exemplo de Saída (50 Gerações)

Geração	Fitness	PID $[K_p, K_i, K_d]$
00	0.1823	$[1.24, 0.87, 0.31]$
01	0.3147	$[2.11, 1.03, 0.45]$
02	0.4892	$[2.87, 1.21, 0.58]$
03	0.5934	$[3.14, 1.35, 0.62]$
04	0.6721	$[3.42, 1.48, 0.67]$
05	0.7103	$[3.58, 1.52, 0.71]$
06	0.7389	$[3.67, 1.55, 0.73]$
07	0.7512	$[3.71, 1.57, 0.74]$
08	0.7601	$[3.74, 1.58, 0.75]$
09	0.7634	$[3.75, 1.59, 0.75]$
10	0.7651	$[3.76, 1.59, 0.76]$



Fitness Crescente

O fitness sobe de 0.1823 (geração 0) para 0.7651 (geração 10) – a população converge progressivamente para controladores de melhor qualidade.

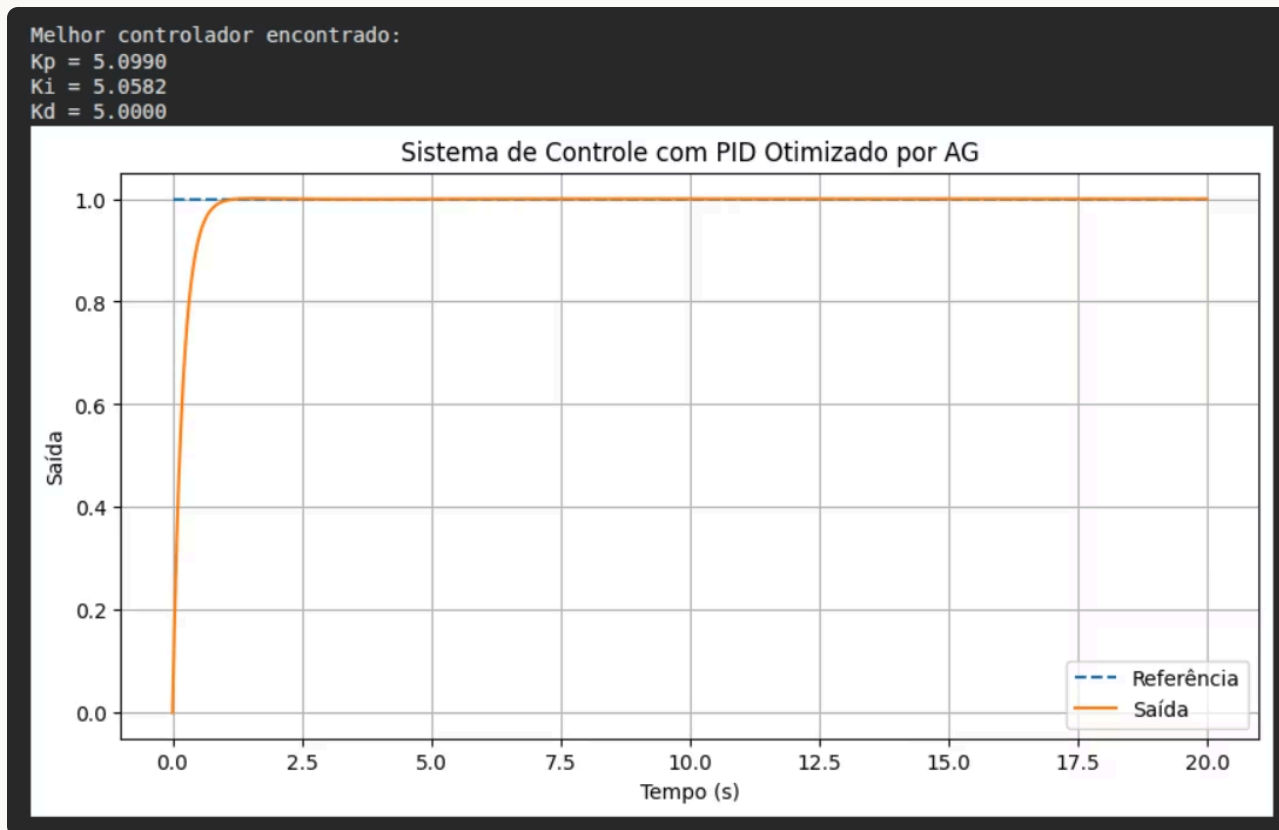


Melhor Indivíduo

$K_p = 3.76 \cdot K_i = 1.59 \cdot K_d = 0.76$ – o controlador PID otimizado pelo AG após 10 gerações de evolução.

Resultado: PID Otimizado pelo AG

Após 50 gerações de evolução, o AG encontrou os ganhos que produzem uma resposta ao degrau rápida, sem overshoot e com erro nulo em regime permanente.



Resposta Rápida

A saída atinge a referência ($y = 1.0$) em menos de 2.5 segundos — tempo de acomodação muito inferior ao sistema sem controle.

Sem Overshoot

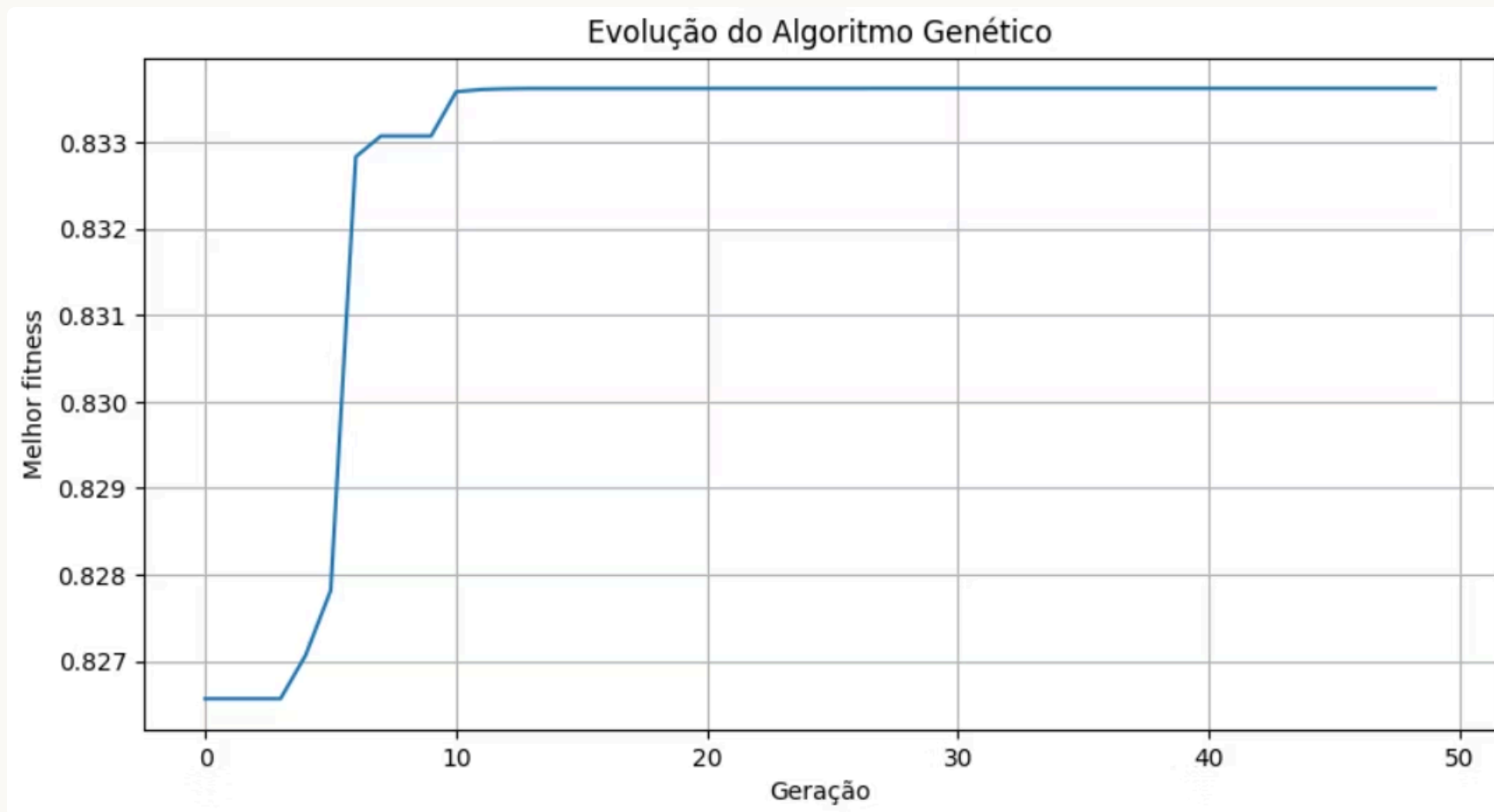
A resposta não ultrapassa o valor de referência — o AG equilibrou K_p , K_i e K_d para evitar oscilações e picos indesejados.

Erro Nulo

Em regime permanente, a saída converge exatamente para $y = 1.0$ — o ganho integral K_i elimina o erro estacionário

Curva de Convergência do AG

O gráfico mostra a evolução do melhor fitness ao longo das 50 gerações – evidenciando a melhoria rápida nas primeiras gerações e a estabilização progressiva.



Referências Bibliográficas

- **JANG, Jyh-Shing Roger; SUN, Chuen-Tsai; MIZUTANI, Eiji.** *Neuro-fuzzy and soft computing: a computational approach to learning and machine intelligence*. Upper Saddle River, NJ: Prentice Hall, 1997. 614 p.
- **HAYKIN, S.** *Neural Networks and Learning Machines*. 3ª ed. Pearson.
- **GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A.** *Deep Learning*. MIT Press, 2016.
- **Nogueira, B. (2026).** *Busca Local e Algoritmos Genéticos* [Slides de aula]. Faculdade de Computação, Universidade Federal de Mato Grosso do Sul